Redis para la API de ventas

Plan de implementación guiada | .NET 10 | Edición 1

Propósito y alcance

Incorporar caché distribuida sobre las consultas existentes de pos_caja.dbo.venta. Este documento es un plan: no instala paquetes, no conecta a Redis, no modifica clases y no cambia la base de datos. Los comandos se muestran para ejecutarlos después, en el orden indicado.

Lo que ya existe

La solución cuenta con API REST, servicio de consultas, contrato IVentaRepository, repositorio SQL con EF Core, pruebas, Swagger UI y configuración SQL mediante variables de entorno. La clave real de una venta es (IdVenta, IdPos). Las consultas disponibles son por clave compuesta y por página.

Resultado esperado

Primera solicitud: consultar Redis; al faltar la entrada, consultar SQL y guardar el resultado. Solicitud repetida con la misma clave: devolver el resultado de Redis. Al vencer la entrada, volver a SQL. Si Redis no está disponible, continuar con SQL, con una demora acotada por la configuración del cliente.

Flujo previsto

VentasController → VentaService → IVentaRepository → CachedVentaRepository.

El decorador consultará RedisJsonCache → IDistributedCache → Redis. Cuando no haya resultado utilizable, llamará al repositorio SQL → PosCajaDbContext → SQL Server.

Decisiones adoptadas

Instancia Redis existente; sin instalar otro servidor. TTL absoluto de 60 segundos. No habrá caché en memoria local, invalidación por eventos, escrituras de ventas ni cambios de esquema. Redis es una copia descartable de resultados, no una réplica automática de la tabla.

Cómo leer las explicaciones por línea

Cada número en las fichas de funciones representa una futura instrucción lógica y explica su objetivo. No son números de líneas de archivos existentes ni código C# listo para pegar. Las declaraciones, validaciones, llamadas, salidas y bloques de errores están descritos en lenguaje natural. Al implementar explicaremos además la sintaxis de cada línea real.

Bibliotecas y efecto en la arquitectura

Qué se añade y qué conserva su responsabilidad

Biblioteca elegida

Microsoft.Extensions.Caching.StackExchangeRedis 10.0.14: proveedor de Redis para IDistributedCache, compatible con net10.0 [1,2]. IDistributedCache define operaciones asíncronas para leer, escribir y quitar entradas. StackExchange.Redis es el cliente subyacente; también aporta ConfigurationOptions y tipos de errores. No crearemos una conexión por petición.

Serialización y configuración

System.Text.Json convierte VentaDto y PaginaResultado<VentaDto> a bytes UTF-8 y viceversa. Microsoft.Extensions.Options valida RedisCacheSettings y lo entrega mediante IOptions. El proveedor administra su conexión compartida. No necesitamos Scrutor ni otro generador de OpenAPI.

Proyecto nuevo

PosCaja.Infrastructure.Redis será una biblioteca net10.0 que referencia PosCaja.Application. Contendrá RedisCacheSettings, su validador, VentaCacheKeys, RedisJsonCache, CachedVentaRepository y DependencyInjection. No referenciará la implementación SQL, controladores ni archivos de configuración.

Domain y Application

Venta conserva todos sus campos. IVentaRepository, IVentaService, VentaDto y PaginaResultado mantienen sus contratos. VentaService conserva la validación de página mínima 1 y tamaño 1 a 100. La aplicación no conocerá IDistributedCache, TLS ni claves Redis.

Infraestructura SQL

Las consultas, proyección, AsNoTracking, precisión decimal y orden por IdVenta/IdPos no cambian. Solo adaptaremos el registro de dependencias para ofrecer una instancia SQL identificada por la clave de servicio 'sql', accesible por la composición de la API.

API y pruebas

Program asociará la sección Redis a las opciones y compondrá el decorador con el repositorio SQL. Controllers, rutas, JSON y Swagger conservarán sus contratos. PosCaja.Tests agregará pruebas de caché y una referencia a la biblioteca nueva. Las pruebas HTTP actuales seguirán sustituyendo IVentaRepository.

Dockerfile y documentación

El Dockerfile deberá copiar el csproj de Redis antes del restore para resolver la nueva referencia. README y ejemplos operativos describirán variables, TTL, fallos y pruebas. No se publicará ni se hará push automáticamente por completar una etapa.

Etapa 1: crear y referenciar la biblioteca

Comandos futuros; ejecutar desde la raíz de la solución

1. Crear el proyecto

```
dotnet new classlib -n PosCaja.Infrastructure.Redis -f net10.0
```

Propósito: Crear la biblioteca y su csproj. 'classlib' selecciona una biblioteca; '-n' define el nombre y carpeta; '-f' selecciona .NET 10. La clase de ejemplo Class1 se retirará al crear las clases reales.

2. Incorporarlo a la solución

```
dotnet sln Api.Redis.slnx add PosCaja.Infrastructure.Redis/PosCaja.Infrastructure.Redis.csproj
```

Propósito: Permitir que build y las herramientas de Visual Studio incluyan el proyecto. No crea una referencia entre bibliotecas.

3. Referenciar los contratos

```
dotnet add PosCaja.Infrastructure.Redis/PosCaja.Infrastructure.Redis.csproj reference
PosCaja.Application/PosCaja.Application.csproj
```

Propósito: Permitir implementar IVentaRepository y utilizar los DTO existentes. No duplicaremos los modelos.

4. Incorporar el proveedor

```
dotnet add PosCaja.Infrastructure.Redis/PosCaja.Infrastructure.Redis.csproj package
Microsoft.Extensions.Caching.StackExchangeRedis --version 10.0.14
```

Propósito: Agregar una versión explícita del proveedor y restaurar sus dependencias. Esta acción instala un cliente .NET, no el servidor Redis. Revisar las advertencias NuGet antes de continuar.

5. Referencias de API y pruebas

```
dotnet add Api.Redis/Api.Redis.csproj reference PosCaja.Infrastructure.Redis/PosCaja.Infrastructure.Redis.csproj

dotnet add PosCaja.Tests/PosCaja.Tests.csproj reference PosCaja.Infrastructure.Redis/PosCaja.Infrastructure.Redis.csproj
```

Propósito: La API podrá registrar el decorador y las pruebas podrán verificarlo.

6. Verificar el punto de partida

```
dotnet build Api.Redis.slnx

dotnet test Api.Redis.slnx
```

Propósito: Confirmar que la estructura compila y las pruebas existentes siguen pasando. Al terminar esta etapa Redis todavía no participará en las consultas.

Etapa 2: variables de entorno

Configuración separada de SQL Server y sin secretos en Git

Opciones propuestas

Redis__Enabled: false inicialmente; activar después de validar el entorno.

Redis__Host: obligatorio al activar; nombre DNS o dirección del Redis existente.

Redis__Port: 6379 por defecto; sustituir por el puerto real.

Redis__Database: 0 por defecto; esta primera versión soporta únicamente 0, compatible con Redis Cluster.

Redis__User y Redis__Password: opcionales según la autenticación del servidor; si se indica User debe existir Password.

Transporte, expiración y límites

Redis__Ssl: true por defecto; usar false solo si el Redis local de desarrollo no ofrece TLS.

Redis__SslHost: opcional; identidad TLS si el proveedor exige un nombre concreto.

Redis__TtlSeconds: 60.

Redis__ConnectTimeoutMilliseconds: 1000.

Redis__AsyncTimeoutMilliseconds: 1000.

Redis__KeyPrefix: obligatorio al activar, por ejemplo Api.Redis:Development:pos_caja:venta:v1:. Debe distinguir ambiente y origen SQL.

Carga de valores

WebApplication.CreateBuilder ya incorpora las variables de entorno. '__' se convierte en ':'; Program leerá la sección Redis y la asociará a RedisCacheSettings. No utilizaremos Environment.GetEnvironmentVariable dentro del repositorio ni leeremos variables persistentes del registro de Windows desde la aplicación.

Comandos de desarrollo

```
dotnet user-secrets set "Redis:Host" "localhost" --project Api.Redis

dotnet user-secrets set "Redis:Enabled" "false" --project Api.Redis
```

Propósito: Guardar configuración local fuera del repositorio. 'localhost' es solo un ejemplo: hay que usar el endpoint real. Para TLS, credenciales y prefijo se usa el mismo formato Redis:Propiedad. No colocar contraseñas en este documento.

Precedencia y reinicio

Las variables de entorno prevalecen sobre User Secrets y appsettings. Las variables del perfil de lanzamiento sobrescriben los valores heredados para ese proceso. Reiniciar Visual Studio tras cambiar variables persistentes. Un contenedor requiere inyección explícita. No reutilizar las opciones SQL TrustServerCertificate para Redis.

Información necesaria antes de activar

Confirmar host, puerto, TLS, autenticación, entorno y permisos Redis. La existencia de una instancia no determina estos datos. El plan no presume credenciales ni los descubre de fuentes ajenas. El proveedor de producción deberá ofrecer certificado confiable; no se desactivará su validación.

Funciones de configuración: instrucción por instrucción

RedisCacheSettings y RedisCacheSettingsValidator

Declaración de RedisCacheSettings

1. Declarar una clase pública sellada: representa solo opciones de Redis.
2. Definir SectionName con 'Redis': evita repetir el nombre de sección.
3. Declarar Enabled=false: permite desplegar la biblioteca manteniendo SQL directo.
4. Declarar Host, Port, Database, User y Password: describen el destino y autenticación.
5. Declarar Ssl y SslHost: describen el transporte seguro.
6. Declarar KeyPrefix y TtlSeconds: aíslan entradas y limitan su vida.
7. Declarar los dos timeouts: limitan la espera del cliente.
8. No sobrescribir ToString para mostrar propiedades: evitar exposición accidental de secretos.

Validate(name, options)

1. Si Enabled es false, devolver éxito: no exigir Redis para ejecutar SQL directo.
2. Crear una lista de errores sin valores de configuración.
3. Comprobar Host no vacío y Port entre 1 y 65535.
4. Exigir Database=0 para este alcance inicial.
5. Exigir KeyPrefix no vacío y terminado en ':'.
6. Exigir TtlSeconds entre 1 y 3600.
7. Exigir timeouts entre 100 y 30000 milisegundos.
8. Si User existe, exigir Password; permitir contraseña sin User para autenticación del usuario predeterminado.
9. Devolver todos los errores juntos o éxito. No abrir una conexión durante la validación.

BuildConfigurationOptions()

1. Obtener opciones validadas: nunca concatenar una cadena con contraseña.
2. Crear ConfigurationOptions de StackExchange.Redis.
3. Añadir Host y Port a EndPoints y fijar DefaultDatabase=0.
4. Asignar User/Password solo cuando existan.
5. Aplicar Ssl y SslHost conservando la validación TLS predeterminada.
6. Aplicar ConnectTimeout y AsyncTimeout.
7. Fijar ConnectRetry=0, AbortOnConnectFail=false y BacklogPolicy.FailFast para evitar colas de operaciones durante una desconexión.
8. Devolver el objeto al proveedor; nunca registrarlo completo.

Qué significa cada límite

ConnectTimeout acota intentos de conexión; AsyncTimeout acota operaciones Redis asíncronas. No son el timeout del comando SQL ni un SLA HTTP exacto: DNS, TLS y planificación pueden añadir tiempo. La primera versión no implementará un circuit breaker; mediremos la demora durante fallos.

Criterio de salida

Configuración inválida y Redis activado: el arranque falla con errores seguros. Configuración válida pero servidor caído: el diseño permite iniciar y recurrir a SQL al solicitar datos. Redis desactivado: no se intenta conexión.

Etapa 3: composición de dependencias

Evitar ciclos y mantener bibliotecas reutilizables

Ajuste de AddPosCajaSqlServer

1. Mantener el registro Scoped de PosCajaDbContext.
2. Registrar IVentaRepository → VentaRepository como servicio Scoped con clave 'sql'.
3. Mantener un registro IVentaRepository sin clave que resuelva el servicio 'sql': conserva el funcionamiento SQL cuando no se usa Redis.
4. Mantener validación y construcción de la conexión SQL existentes. No agregar referencias Redis al proyecto SQL.

Nueva función AddPosCajaRedis

Entradas: acción para configurar RedisCacheSettings y fábrica Func<IServiceProvider, IVentaRepository> para obtener el repositorio original.

1. Validar que ambos argumentos existan.
2. Registrar opciones, validador y ValidateOnStart.
3. Registrar AddStackExchangeRedisCache y configurar sus opciones desde RedisCacheSettings mediante el sistema Options.
4. Entregar ConfigurationOptions al proveedor. Dejar InstanceName vacío: el prefijo lo incluirán las claves, una sola vez.
5. Registrar RedisJsonCache como Singleton: solo utiliza servicios compartibles, no DbContext.
6. Sustituir el registro sin clave de IVentaRepository mediante una fábrica Scoped.
7. En esa fábrica, obtener opciones y el repositorio SQL proporcionado.
8. Si Enabled=false, devolver SQL directamente; no resolver el proveedor Redis.
9. Si Enabled=true, construir CachedVentaRepository con SQL, RedisJsonCache, opciones y logger.
10. Devolver IServiceCollection para mantener el patrón de registro actual.

Cambios de Program

1. Registrar SQL primero, como hoy.
2. Registrar Redis después y asociar la sección Redis con Bind.
3. Proporcionar una fábrica que resuelva GetRequiredKeyedService<IVentaRepository>('sql').
4. Conservar IVentaService → VentaService Scoped.
5. Mantener controladores, ProblemDetails, Swagger y rutas existentes.
6. No llamar Connect ni esperar un PING durante el arranque web.

Por qué importa la fábrica

Si CachedVentaRepository solicita IVentaRepository sin distinguir el original, el contenedor podría intentar construir el mismo decorador recursivamente. La clave 'sql' identifica el repositorio original. La biblioteca Redis recibe una abstracción y no depende de VentaRepository ni del proyecto SQL.

Ciclos de vida

DbContext, VentaRepository, CachedVentaRepository y VentaService: Scoped, uno por solicitud. RedisJsonCache y el proveedor IDistributedCache: compartidos. No registrar el decorador como Singleton porque retendría el repositorio SQL y su contexto.

Verificación

Resolver IVentaRepository con Redis activo debe producir el decorador; con Redis inactivo, SQL. Las pruebas deben verificar que ambas ramas se construyen sin ciclos y que no se abre Redis al estar desactivado.

Claves y representación de resultados

Etapa 4: identidad, aislamiento y JSON

VentaCacheKeys.ForVenta

1. Recibir prefijo, Guid idVenta e int idPos.
2. Formatear el Guid con formato N: 32 caracteres hexadecimales sin guiones.
3. Formatear idPos con cultura invariable.
4. Concatenar 'item:', Guid, ':' e idPos después del prefijo.
5. Devolver la clave. Incluir ambos identificadores evita mezclar puntos de venta.

VentaCacheKeys.ForPage

1. Recibir prefijo, pagina y tamanoPagina ya validados por VentaService.
2. Formatear ambos enteros con cultura invariable.
3. Añadir 'page:{pagina}:size:{tamanoPagina}' al prefijo.
4. Devolver la clave. No incluir filtros que la API no soporta; cualquier filtro futuro deberá añadirse a la identidad.

Ejemplos conceptuales

Api.Redis:Development:pos_caja:venta:v1:item:11111111111111111111111111111111:2

Api.Redis:Development:pos_caja:venta:v1:page:1:size:50

Estas son claves, no comandos ejecutados. Development y Production no deben compartir el prefijo; tampoco dos bases distintas.

Qué se serializa

Para una venta: VentaDto, nunca la entidad rastreada ni el DbContext. Para una página: PaginaResultado<VentaDto> completo, incluidos Items, Pagina, TamanoPagina y TotalRegistros. Los importes permanecen decimal, el total long y Supervisor nullable. FechaVenta conserva su semántica actual, sin conversión artificial a UTC.

Opciones JSON

Definir una instancia compartida de JsonSerializerOptions con valores web coherentes con la API. No mutarla después del primer uso. SerializeToUtf8Bytes escribe el payload; Deserialize<T> lo recupera. Las claves tienen versión v1 para permitir futuras incompatibilidades sin leer formatos antiguos.

Resultado interno de lectura

CacheReadResult<T> tendrá Found y Value. Found distingue una entrada ausente de un valor utilizable; no utilizar default(T) como único indicador. Null deserializado se considera ausencia. Los DTO guardados no serán null.

Consistencia aceptada

TTL absoluto: 60 segundos desde la escritura Redis, sin renovación por lectura. No es una garantía exacta de antigüedad desde el cambio SQL: la consulta y las escrituras concurrentes añaden desfase. Páginas y ventas individuales pueden reflejar momentos distintos. El conteo y los elementos ya son consultas SQL separadas.

RedisJsonCache: lectura y escritura

Cada instrucción se implementará y explicará durante la etapa

Constructor

1. Recibir IDistributedCache, IOptions<RedisCacheSettings> y ILogger<RedisJsonCache>.
2. Guardar dependencias de solo lectura.
3. Preparar las opciones JSON compartidas y la duración TTL.
4. No abrir conexiones ni guardar datos por solicitud en campos.

TryGetAsync(key, cancellationToken)

1. Comprobar cancelación del solicitante.
2. En un bloque limitado a Redis, esperar GetAsync(key, token).
3. Si Redis devuelve null, devolver Found=false y registrar miss en nivel Debug.
4. Si existen bytes, intentar Deserialize<T>.
5. Si el JSON es inválido o el valor es null, registrar invalid y devolver ausencia; no exponer el contenido.
6. Si es válido, devolver Found=true con el DTO.
7. Capturar fallos del cliente Redis (conexión, timeout o servidor) como ausencia y registrar advertencia segura.
8. No absorber OperationCanceledException ni errores arbitrarios de programación.

TrySetAsync(key, value, cancellationToken)

1. Verificar cancelación y que value no sea null.
2. Serializar el DTO a UTF-8; el esquema conocido debe ser serializable.
3. Crear DistributedCacheEntryOptions con AbsoluteExpirationRelativeToNow de 60 segundos, tomada de opciones. No establecer SlidingExpiration.
4. Esperar SetAsync(key, bytes, entryOptions, token).
5. Si la escritura funciona, finalizar sin alterar la respuesta SQL.
6. Si Redis falla, registrar write_failed y finalizar: SQL ya produjo una respuesta válida.
7. Un JsonException esperado se registra como fallo de caché; errores de programación inesperados no se ocultan.
8. Propagar cancelación de la petición.

Corrupción y eliminación

No borraremos automáticamente una entrada corrupta antes de consultar SQL: un borrado podría eliminar una escritura concurrente válida. La respuesta SQL la reemplazará si es cacheable; de lo contrario vencerá por TTL. RemoveAsync se reserva para una futura invalidación explícita.

Registros y límites

Registrar operación, tipo de consulta, resultado, duración y tipo de error. No registrar credenciales, configuración de conexión, payload, excepción completa que pueda incluir datos sensibles ni valores de ventas. Los mensajes warning deben permitir detectar autenticación o TLS fallidos aunque la API continúe con SQL.

Una distinción esencial

Los try/catch de Redis rodean exclusivamente Redis. La llamada a SQL se hará fuera de ellos. Un error de base de datos no se convertirá en 'cache miss' ni en una respuesta vacía.

ObtenerPorIdAsync: recorrido completo

Etapa 5: primera consulta cacheada

Constructor del decorador

1. Declarar CachedVentaRepository como implementación de IVentaRepository.
2. Recibir el repositorio original, RedisJsonCache, opciones y logger.
3. Guardarlos sin abrir conexiones. El repositorio original lo decide la fábrica de composición; no se resuelve de nuevo IVentaRepository.

Instrucciones del método

1. Recibir Guid idVenta, int idPos y CancellationToken, conservando la firma actual.
2. Comprobar cancelación para evitar trabajo innecesario.
3. Construir la clave con ForVenta y el prefijo configurado.
4. Esperar TryGetAsync<VentaDto>.
5. Si Found=true, comprobar que IdVenta e IdPos del payload coinciden con la solicitud.
6. Si coinciden, devolver el DTO inmediatamente: no invocar SQL.
7. Si no coinciden, tratar la entrada como inválida y continuar; registrar solo el tipo de incidencia.
8. Esperar inner.ObtenerPorIdAsync con ambos identificadores y el mismo token.
9. Si SQL devuelve null, devolver null sin guardarlo en Redis; el controlador seguirá produciendo 404.
10. Si existe la venta, esperar TrySetAsync con el DTO.
11. Devolver el DTO de SQL incluso si la escritura Redis no pudo realizarse.

Por qué no cachear el 404

La ausencia podría cambiar si otra aplicación crea la venta. En esta versión cada consulta inexistente vuelve a SQL; se prioriza una política simple. El decorador retorna null, no un objeto HTTP ni un DTO vacío.

Ejemplo de verificación

Elegir una clave existente sin divulgar datos en registros. Primera petición: falta Redis, se observa una consulta individual SQL y se crea la entrada. Segunda petición: se devuelve el mismo contrato y no se observa consulta SQL. Repetir con el mismo Guid y otro IdPos para confirmar aislamiento.

Criterio de salida

Prueba de hit sin llamada SQL; miss con una sola llamada SQL; null no guardado; clave compuesta correcta; token propagado; fallo Redis con retorno SQL; fallo SQL propagado; JSON con decimal y Supervisor null preservados.

Lo que no cambia en HTTP

GET /api/ventas/{idVenta:guid}/{idPos:int} continúa con 200 o 404 y el mismo esquema JSON. No añadiremos campos como 'fromCache' al DTO. La procedencia se observará mediante registros y pruebas.

ObtenerPaginaAsync: páginas independientes

Etapa 6: caché del listado completo

Instrucciones del método

1. Recibir pagina, tamanoPagina y CancellationToken con los valores predeterminados existentes.
2. Mantener VentaService como punto de validación; la petición inválida no debe alcanzar el repositorio.
3. Comprobar cancelación.
4. Crear ForPage con los dos parámetros normalizados.
5. Esperar TryGetAsync<PaginaResultado<VentaDto>>.
6. Si existe, comprobar que Pagina y TamanoPagina coincidan, Items no sea null, su cantidad no supere el tamaño solicitado y TotalRegistros no sea negativo.
7. Si el contenido es válido, devolver toda la página, incluido su total, sin SQL.
8. Si falta o es inválido, esperar inner.ObtenerPaginaAsync con los mismos argumentos.
9. Esperar TrySetAsync con el resultado completo, incluso si Items está vacío.
10. Devolver el resultado de SQL. No recalcular el total al recuperar un hit.

Por qué guardar el total

Guardar únicamente Items obligaría a ejecutar COUNT_BIG en cada petición y reduciría el beneficio de la caché. La respuesta almacenada es exactamente la que el repositorio SQL generó en ese momento.

Página vacía frente a venta inexistente

Una página sin elementos sigue siendo un resultado válido 200 y puede tener un total mayor que cero cuando se solicita una página posterior a la última. Se guarda por 60 segundos. Una venta individual inexistente retorna null y no se guarda.

Ejemplos de identidad

pagina=1,tamanoPagina=50 y pagina=2,tamanoPagina=50 son entradas distintas. pagina=1,tamanoPagina=10 es otra entrada. Una petición sin parámetros equivale a pagina=1,tamanoPagina=50 y debe producir la misma clave.

Carga SQL y concurrencia

En miss se ejecutan las dos consultas actuales: conteo y página. En hit no se ejecuta ninguna. No se paralelizarán consultas sobre el mismo DbContext. Dos solicitudes simultáneas que no encuentren la clave pueden consultar SQL ambas; la versión inicial no añade bloqueos distribuidos ni protección contra avalanchas.

Criterio de salida

Validar aislamiento por tamaño y página; caché de página vacía; igualdad de JSON entre miss e hit; conservación de TotalRegistros como long; orden del SQL original intacto; validaciones HTTP anteriores intactas.

Comandos para observar Redis

Etapa 7: inspección controlada de la instancia existente

Conectar y verificar

```
redis-cli -h HOST -p PUERTO --tls --user USUARIO --askpass
```

Propósito: Abrir el cliente con TLS y solicitar la contraseña sin colocarla en la línea de comandos. HOST, PUERTO y USUARIO son marcadores. Omitir --user si no hay ACL con usuario; omitir --askpass si no hay autenticación. Usar --cacert cuando el entorno tenga una CA privada. Omitir --tls solo para un servidor local sin TLS.

Comprobar respuesta

```
PING
```

Propósito: Una respuesta PONG comprueba disponibilidad básica. No verifica permisos de caché completos ni conexión SQL.

```
SELECT 0
```

Propósito: Seleccionar la base lógica inicial en Redis standalone. En Redis Cluster se usa únicamente 0 y no se envía SELECT.

Localizar entradas sin barrer de golpe

```
SCAN 0 MATCH Api.Redis:Development:pos_caja:venta:v1:* COUNT 100
```

Propósito: Recorrer claves por lotes. Repetir SCAN con el cursor devuelto hasta que vuelva a 0. COUNT es una sugerencia; puede devolver cero elementos o duplicados. No usar KEYS * en una instancia compartida [3].

Inspeccionar tipo y expiración

```
TYPE CLAVE_COMPLETA
```

Propósito: Averiguar el tipo Redis de una entrada; CLAVE_COMPLETA se sustituye por una clave real.

```
TTL CLAVE_COMPLETA
```

Propósito: Ver segundos restantes; -2 significa que no existe y -1 que no tiene expiración. Una entrada creada por este plan debe expirar. Redis puede expulsarla antes por presión de memoria [4].

Leer el contenido solo durante pruebas

```
HGET CLAVE_COMPLETA data
```

Propósito: El proveedor de Microsoft puede almacenar el payload y metadatos en un hash. Usar este comando únicamente después de confirmar TYPE=hash y el campo data de la versión instalada. GET no funciona sobre un hash. Este formato es interno: la aplicación usará IDistributedCache, no comandos de hash [5]. El contenido son datos reales; no copiarlo a registros públicos.

Repetir una prueba de miss

```
DEL CLAVE_COMPLETA
```

Propósito: Eliminar solamente la entrada de prueba para forzar una lectura SQL posterior. Es una operación de escritura; ejecutarla solo sobre una clave propia de desarrollo. No ejecutar FLUSHALL ni FLUSHDB.

Prueba de expiración

Llamar dos veces a Swagger, observar hit/miss y TTL. Volver a leer antes de 60 segundos: TTL no debe renovarse. Esperar a que venza y repetir: deberá volver a SQL. No es necesario detener ni reiniciar Redis.

Pruebas automatizadas y aceptación

Etapa 8: demostrar comportamiento, no solo velocidad

Estructura de las pruebas

Añadir CachedVentaRepositoryTests, RedisJsonCacheTests, VentaCacheKeysTests y RedisCacheSettingsTests. Usar xUnit. Un IDistributedCache de prueba almacenará bytes y opciones y permitirá simular fallos; un IVentaRepository de prueba contará llamadas. No introducir dependencias a Redis real en la suite predeterminada.

Casos obligatorios

01. Hit individual: cero llamadas SQL y DTO equivalente.
02. Miss individual: una consulta SQL, serialización y TTL de 60 segundos.
03. Misma venta con distintos IdPos: claves diferentes.
04. Venta inexistente: null y ninguna escritura de caché.
05. Hit de página: sin conteo ni lectura SQL.
06. Páginas y tamaños distintos: sin colisiones.
07. Página vacía: se guarda y retorna correctamente.
08. Fallo Redis al leer: se consulta SQL.
09. Fallo Redis al escribir: se conserva el resultado SQL.
10. Fallo SQL: se propaga al manejador HTTP.
11. JSON corrupto, null o identidad incompatible: volver a SQL.
12. Cancelación: se propaga, no se confunde con una caída Redis.

Configuración y composición

13. Enabled=false: repositorio SQL directo y ninguna operación Redis.
14. Enabled=true sin Host/prefijo: error de configuración seguro.
15. Configuración TLS, autenticación y timeout correctamente aplicada.
16. DI sin ciclos y contexto Scoped.
17. Opciones de TTL absoluto presentes, sin expiración deslizante.
18. Decimal, long, Guid, fecha y null sobreviven a serialización.
19. HTTP 200/400/404/500 y documento OpenAPI mantienen sus contratos.

Comandos de comprobación

```
dotnet test Api.Redis.slnx
```

Propósito: Ejecutar toda la suite antes de dar la etapa por terminada.

```
dotnet test Api.Redis.slnx --collect:"XPlat Code Coverage"
```

Propósito: Recopilar cobertura para identificar ramas de error no ejercitadas; el repositorio no fija porcentaje mínimo.

Integración real separada

Con Redis y SQL configurados, comprobar escritura, lectura, TTL real y recuperación. Simular una conexión Redis fallida en un proceso de prueba mediante un puerto local confirmado cerrado; no apagar una instancia compartida. Verificar HTTP 200 con SQL disponible y medir la demora. Restaurar la configuración después.

Qué no prueba un doble

Un caché simulado puede comprobar las opciones TTL, pero no la expiración del servidor, TLS, ACL, reconexión o latencia de red. Esos resultados deben informarse por separado, sin afirmar cobertura de integración por pasar únicamente las pruebas unitarias.

Secuencia de trabajo y revisión por etapas

Implementar después, con explicación y evidencia en cada entrega

Orden propuesto

Etapa 1: crear biblioteca y referencias. Salida: build y suite base.

Etapa 2: definir opciones y validar variables. Salida: configuración segura, activación todavía deshabilitada.

Etapa 3: componer SQL y decorador sin ciclos. Salida: DI correcta y SQL directo con Enabled=false.

Etapa 4: claves y RedisJsonCache. Salida: serialización, TTL y fallos probados.

Etapa 5: consulta por clave. Salida: hit/miss y 404 correctos.

Etapa 6: paginación. Salida: páginas independientes, total incluido.

Etapa 7: observación real con redis-cli y Swagger. Salida: evidencia de caché y expiración.

Etapa 8: regresión completa, documentación y revisión final.

Qué explicaré al implementar cada archivo

1. Responsabilidad y capa.
2. Dependencias y por qué son necesarias.
3. Cada declaración, parámetro y tipo de retorno.
4. Cada instrucción y qué resultado produce.
5. Cada rama de error y qué no debe capturar.
6. La prueba que demuestra su comportamiento.
7. Los comandos ejecutados y sus resultados.

No se avanzará silenciosamente sobre una etapa fallida.

Matriz de fallos

Redis disponible + hit válido: respuesta caché, aunque SQL esté temporalmente caído.

Redis disponible + miss: SQL; si SQL falla, error HTTP como hoy.

Redis caído + SQL disponible: respuesta SQL; advertencia de caché.

Redis caído + SQL caído: error HTTP, nunca lista vacía falsa.

Configuración Redis inválida + Enabled=true: detener arranque.

Solicitud cancelada: cancelar la operación, no iniciar trabajo adicional.

Límites que aceptamos

No habrá sincronización inmediata cuando otros sistemas modifiquen ventas. No habrá borrado masivo por prefijo en cada consulta. No se cacheará toda la tabla en un único objeto. No se garantiza que dos peticiones concurrentes produzcan una sola lectura SQL. No habrá reintentos manuales ilimitados.

Criterio de aprobación final

El usuario podrá explicar de dónde sale cada respuesta, demostrar que el hit evita SQL, conocer la antigüedad posible y desactivar Redis sin modificar código. Los endpoints actuales y el SQL conservarán su comportamiento contractual.

Activación, reversión y referencias

Condiciones para pasar del plan a una implementación verificable

Activación controlada

Primero ejecutar con Redis__Enabled=false y validar que SQL funciona. Configurar el endpoint existente, autenticación, TLS y un prefijo exclusivo de desarrollo. Activar Redis y reiniciar el proceso. Probar una venta y una página conocidas, observar TTL y confirmar hit. No habilitar inicialmente sobre producción ni sobre otro origen SQL con el mismo prefijo.

Reversión

Establecer Redis__Enabled=false y reiniciar la API. La composición devolverá el repositorio SQL directo. Las entradas existentes vencerán por TTL; no hace falta limpiar toda la instancia. El reinicio aplica también a cambios de credenciales, prefijo y endpoint.

Operación y observabilidad

Monitorizar aciertos, ausencias, errores de lectura/escritura y latencia. Una respuesta 200 puede ocultar que Redis esté caído porque SQL funciona: los registros deben permitir detectarlo. Antes de producción, medir carga SQL bajo caída Redis, revisar memoria/evicción del servidor y conceder a la cuenta de la aplicación solo los permisos necesarios.

Puntos pendientes del entorno

Antes de ejecutar la etapa 2 se necesitan host, puerto, TLS, autenticación y permiso para una clave de prueba. No se necesitan contraseñas en el chat: se inyectarán mediante variables o secretos. La política ACL debe permitir las operaciones que utiliza el proveedor de Microsoft; no basta autorizar GET y SET si su almacenamiento usa hashes.

Referencias oficiales consultadas

[1] Microsoft: caché distribuida en ASP.NET Core 10.

https://learn.microsoft.com/en-us/aspnet/core/performance/caching/distributed?view=aspnetcore-10.0

[2] NuGet: Microsoft.Extensions.Caching.StackExchangeRedis 10.0.14.

https://www.nuget.org/packages/Microsoft.Extensions.Caching.StackExchangeRedis/10.0.14

[3] Redis: SCAN.

https://redis.io/docs/latest/commands/scan/

[4] Redis: TTL.

https://redis.io/docs/latest/commands/ttl/

[5] Microsoft: implementación RedisCache, referencia para el formato interno.

https://github.com/dotnet/aspnetcore/blob/main/src/Caching/StackExchangeRedis/src/RedisCache.cs

[6] StackExchange.Redis: opciones de configuración.

https://stackexchange.github.io/StackExchange.Redis/Configuration.html

Estado de esta entrega

Solo se genera este documento. No se han creado clases Redis, agregado paquetes al proyecto, ejecutado comandos Redis ni cambiado las variables del sistema. El trabajo de la siguiente sesión comenzaría por la etapa 1.